

# AI-assisted security review

Mempire · three Solana programs · 4,573 lines of Rust

**This is not a professional security audit.**

It is a structured review carried out by AI agents. Every finding was verified against the deployed programs and the serious fixes were proven on devnet by carrying out the attack and showing it no longer pays. It is evidence of diligence — not a guarantee of safety, and not a substitute for a human audit by a specialist firm. The programs remain unaudited in the industry sense of that word.

|                                  |                                                  |
|----------------------------------|--------------------------------------------------|
| Method                           | solana-auditor · 9 parallel agents · deep mode   |
| Scope                            | mempire · mempire-rollup · mempire-amm           |
| Framework                        | Anchor 0.32.1 · MagicBlock ER 0.16.2 · VRF 0.4.1 |
| Confirmed and fixed              | 7                                                |
| Confirmed, deferred with reasons | 4                                                |
| Rejected after verification      | 4                                                |

Agents produce hypotheses, not results. Four confidently-worded findings in this review were wrong on a careful reading of the source and were rejected — which is the honest measure of how much weight to put on the rest.

## What was reviewed, and how

Nine agents examined every non-test Rust file in the workspace, each from a different angle: attack-vector scanning, math and precision, access control, economic security, execution tracing, invariants, peripheral files, first principles, and a deeper Solana-protocol pass.

Every surviving finding was re-read against the actual source before being accepted. Fixes were deployed to devnet and demonstrated — in the most serious case, by performing the attack and showing it no longer pays.

## Fixed

### 1 · A losing player could take the whole pot

**CRITICAL** · `mempire::claim_timeout`

**claim\_timeout** decides who may claim by reading the match log, guarded on the log being owned by this program. Under the launch build the log is delegated to a MagicBlock rollout for the whole match, so it is owned by the delegation program and that read yields nothing — indistinguishable from “no log was ever created”. Both fell through to *“genuine abandonment, either may claim”*.

The protection disabled itself in exactly the case its own comment describes: a loser who never claims could call this at the deadline and take the pot.

**Fixed.** A log that exists but cannot be read is now treated as a dispute — both stakes refunded, no rake. It cannot reward a stall, and cannot rob the winner of a genuinely vanished opponent of more than the upside.

**Demonstrated** on devnet against match #83, authentically stuck delegated. The seat that had just lost 1–0 called `claim_timeout`:

```
before winner A 1.083505 · loser B 0.729484
after winner A 1.133505 · loser B 0.779479
deltas A +0.050000 · B +0.049995
state Settled, winner 2 (void — both refunded)
```

Before the fix, that call paid the loser the 0.09 pot and left the winner with nothing.

### 2 · Either seat could freeze the match and force a timeout

**HIGH** · `mempire::checkpoint`

Checkpoints wrote the same tick cursor that card plays are gated on, and the value was unbounded. A losing seat could checkpoint at the maximum, after which every play by *either* seat failed for the rest of the match — turning a lost position into a forced timeout, which is a payout path rather than a loss.

**Fixed.** A checkpoint attests a state hash and no longer moves the play cursor.

### 3 · One seat could consume the whole input log

**HIGH** · `mempire::play_card`

The 128-entry play log was a single budget shared by both seats, and the ordering guard is non-strict — so one seat could spend all of it at a single tick. The opponent's inputs then had nowhere to go, the seats necessarily disagreed, and the pot voided: strictly better for whoever was losing.

**Fixed.** Each seat gets half the log, counted from the entry's own seat field.

### 4 · A stranger could burn the winner's reward

**MEDIUM** · `mempire::settle_from_log`

The settler said “either player may settle” in a comment and enforced nothing, while the reward accounts are optional and a mismatch is skipped rather than raised. Any third party could settle a finished match with the reward omitted: the pot paid correctly, the match became settled, and the winner’s 50 \$MEMPIRE became permanently unpayable.

**Fixed.** The settler is pinned to the two seats — a stranger is refused, a real player passes through.

**5 · The randomness guard failed open****HIGH (v3)** · `mempire_rollup::is_ephemeral_queue`

The guard gated the *strict* branch behind a feature that is opt-in, whose crate default is empty, and which no build line in the repo passes for this program. An ordinary build therefore accepted the SDK’s test queue — whose fulfiller is documented as usable by anyone — under a program id registered for mainnet. Chest tiers would have been chosen by whoever answered the oracle first.

**Fixed.** The polarity is reversed: production-only is what a plain build gives you, and the test queue is an explicit opt-in for local development.

**7 · The rollup’s match log was not bound to a real match****CRITICAL (v3)** · `mempire_rollup::init_log`

`init_log` took the match id and the two player keys as instruction data and checked only that the payer was one of the keys they had just supplied. Nothing tied the log to an escrowed match, so two self-owned wallets could invent one, “win” it, and mint the chest entitlement that exists precisely to make a chest cost a win — and, because match ids are sequential and the seeds public, could squat the log of a genuine match and deny it.

**Fixed.** The seats are read from the money program’s own match account, pinned three ways: owned by that program, at that program’s address for this match id, and re-checked field by field. The supplied keys survive as a checked assertion rather than a source of truth, and the match must be active.

**Demonstrated** on devnet against a freshly created match:

```
fabricated match, self-chosen seats REFUSED
real match, an imposter in one seat REFUSED
real match, the seats the chain says ACCEPTED
```

**6 · A player could name the validator that authors their own rewards****HIGH (v3)** · `mempire_rollup::delegate_chests`

A caller-supplied validator was forwarded straight into the delegation config. A delegated account’s authority is what computes and commits its state, so the rail’s owner could choose who authors their entitlements, every chest tier, and the randomness behind it.

**Fixed.** Router-assigned, matching the main program’s equivalent handler.

## Confirmed, deferred — with reasons

These are real. They are not fixed, and the reasons are stated rather than implied.

### The win reward is farmable by self-play

Joining a match rejects the same *key*, not the same person, so two wallets can wash-trade at a net cost of the rake. **Not fixed** because the remedy is a product decision — rate-limit, reprice below rake, or require distinct funding.

### Match rent is never reclaimed

Roughly 0.016 SOL of player rent is stranded per match. **Not fixed** because closing the match account would break the instruction that frees cards still locked to it.

### Selling a card's NFT conveys nothing

Card ownership lives in a field no instruction transfers, so the 1-of-1 is a detached certificate and the seller keeps the playable card. **Not fixed** — NFTs are not in the launch build, and binding the two is a design change.

### The AMM's pool creation is permissionless

Anyone can front-run the canonical pool and fix its fee forever. **Not fixed, and not planned** — \$MEMPIRE launches on Bags and this program is not deployed.

## Rejected after verification

Reported by agents, checked against the source, and found not to hold:

|                                           |                                                    |
|-------------------------------------------|----------------------------------------------------|
| Self-merging a card for a free level      | Both the identity and same-coin guards are present |
| Building a deck from one card eight times | Deck validation rejects a repeated coin            |
| Redirecting fees to an attacker wallet    | Every fee destination is pinned to the treasury    |
| Type confusion in the card migration      | No other account type is the required size         |

## What this review cannot tell you

AI analysis cannot prove the absence of vulnerabilities. It reasons about the code it was shown, and it produced four confidently-worded findings here that were wrong — which is the honest measure of how much weight a reader should put on the rest.

What it is: nine independent passes over every line of the on-chain code, each surviving finding re-derived from source, and the serious ones demonstrated by carrying out the attack and showing the fix defeats it.

What would strengthen it: a human audit by a specialist firm, a bug bounty, and on-chain monitoring. Until then the compensating controls are small stake tiers and a settlement design that voids rather than pays when the two seats disagree — a cheat costs the cheater the match; it cannot take the pot.

Review workflow: solana-auditor, lineage from pashov/skills, adapted for Solana. Full findings and the code they refer to are public at [github.com/nickthelegend/mempire](https://github.com/nickthelegend/mempire).